Jogando Flappy Bird com NEAT Algoritmo Genético

Marcos Eduardo de Souza[1]

[1]Instituto de Computação – Universidade Federal do Rio de Janeiro (UFRJ)
Edifício do Centro de Ciências Matemáticas e da Natureza – Bloco E
Cidade Universitária – Rio de Janeiro – RJ – CEP: 21941-916.

marcoses@ic.ufrj.br

Abstract. *Robotics and the use of artificial agents have long since become commonplace. And a large number of algorithms for generating game agent behaviors were developed during that time. In this context, this article proposes a training strategy to develop autonomous virtual players using the NEAT algorithm to evolve an agent capable of playing the game Flappy Bird. The Algorithm uses an Artificial Neural Network and Genetic Algorithm together with its fitness function, we can maximize the current generation score. Also applying Reinforcement Learning which remembers the state, the action performed in that state and the reward received for the action performed using a single agent and a Deep Q-learning Network.*

Resumo. *A robótica e o uso de agentes artificiais há muito se tornaram comuns. E um grande número de algoritmos para gerar comportamentos de agentes de jogos foram desenvolvidos durante esse período. Nesse contexto, este artigo propõe uma estratégia de treinamento para desenvolver jogadores virtuais autônomos utilizando o algoritmo NEAT para evoluir um agente capaz de jogar o jogo Flappy Bird. O Algoritmo usa uma Rede Neural Artificial e Algoritmo Genético juntamente com sua função de fitness, podemos maximizar a pontuação da geração atual. Aplicando também Aprendizado por Reforço que lembra o estado, a ação realizada naquele estado e a recompensa recebida pela ação realizada usando um único agente e uma Deep Q-learning Network.*

1. Introdução

A técnica de Neuroevolução [Victor de Oliveira 2018] é a evolução artificial da rede neural usando algoritmo genético. É uma técnica para evoluir redes neurais artificiais em problemas de aprendizado não supervisionado. A neuroevolução é usada como parte do aprendizado de reforço, o NEAT desenvolve a rede neural densa nó por nó e altera os valores das conexões e adiciona aleatoriamente outros nós para encontrar a topologia da rede neural que melhor funcione. A ideia principal é apresentar uma alternativa para o algoritmo de backpropagation no cálculo de pesos da rede neural e também uma solução para o problema de arquitetura da rede neural, fazendo com que ambas as coisas sejam calculadas através do algoritmo genético.

2. NEAT

O **NEAT** é uma abreviação para "**NeuroEvolution of Augmenting Topologies**"e é um algoritmo que tem como objetivo identificar padrões gradualmente. Esse algoritmo

aborda alguns desafios enfrentados pelas redes neurais artificiais, como a evolução de peso e topologia. Depois de algumas iterações, o algoritmo começa a se desenvolver de forma exponencial, até se tornar invencível (em jogos simples, como o Flappy Bird, esse processo pode ser muito rápido). Antes de explicar como o NEAT funciona, é importante compreender três conceitos-chave: Rede Neural Artificial (RNA), Algoritmo Genético (AG) e Aprendizado por Reforço (AR). Essas técnicas são utilizadas em conjunto com o NEAT para treinar jogadores virtuais autônomos.

2.1. Rede Neural Artificial

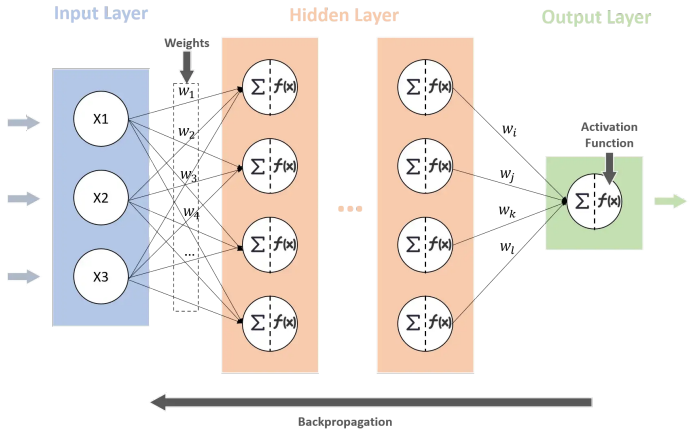


Figura 1. Arquitetura de rede neural artificial. [Cordeiro et al. 2019]

Uma rede neural artificial é um modelo computacional que se inspira no funcionamento do cérebro humano para realizar tarefas de aprendizado e tomada de decisão. Consiste em uma rede interconectada de neurônios artificiais, que processam informações por meio de uma série de camadas de processamento. Cada camada pode ter diversas unidades de processamento, que recebem entradas e realizam operações matemáticas para produzir uma saída. Com o treinamento adequado, as redes neurais podem aprender a reconhecer padrões em dados complexos, como imagens, sons e texto, e realizar tarefas sofisticadas, como reconhecimento de voz, classificação de imagens e previsão de séries temporais.

2.2. Algoritmo Genético

Por definição, é uma técnica de otimização baseada no processo de seleção natural que ocorre na evolução biológica. É uma forma de aprendizado de máquina que usa métodos de busca baseados em populações e gerações para encontrar soluções para problemas complexos. Aqui estão alguns dos principais termos:

- **Função de aptidão(fitness):** Em muitos casos, uma função de aptidão é o mesmo que uma função objetivo. É uma função que recebe um candidato a solução como entrada e gera uma pontuação de aptidão como saída para avaliar a qualidade de cada solução.
- **População:** No AG, a população é o subconjunto de todas as soluções candidatas para um determinado problema.
- **Cromossomo:** Cada solução candidata na população é um cromossomo, às vezes chamado de genoma.

- **Gene:** Cada posição de elemento dentro de um cromossomo é um gene, como uma variável em um problema de otimização.
- **Seleção:** A seleção é o processo de filtrar e reter soluções de acordo com a pontuação de aptidão. Um par de soluções selecionadas pode ser escolhido como pais para reproduzir e propagar seus genes para a geração subsequente por meio de cruzamento(crossover).
- **Crossover:** É a maneira como os cromossomos pais produzem cromossomos filhos. Os genes dos pais são reagrupados com base em um conjunto de pontos de cruzamento para formular novos descendentes.
- **Mutação:** A mutação no GA é um pequeno ajuste aleatório dos genes nos cromossomos.
- **Geração:** A cada iteração, o AG executa um conjunto de operadores genéticos (seleção, cruzamento, mutação, etc.) na população atual para gerar uma geração sucessiva.

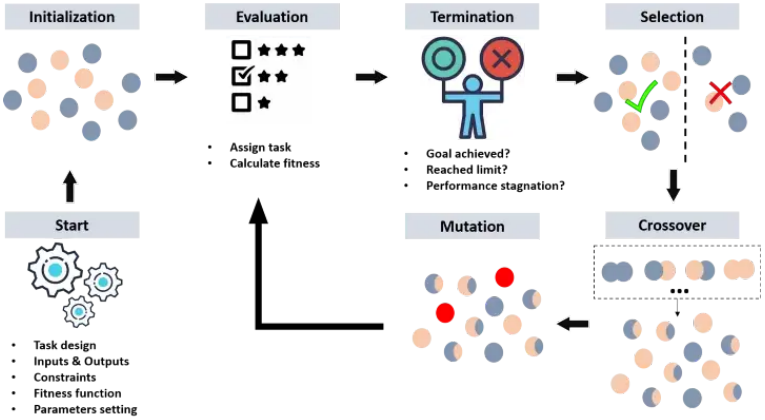


Figura 2. Fluxo de trabalho do algoritmo genético.[Danny Zhu 2020]

2.3. Aprendizado por Reforço

O Aprendizado por Reforço é uma técnica de treinamento em que um agente de inteligência artificial aprende a tomar decisões em um ambiente complexo, recebendo recompensas ou penalidades com base em suas ações. O objetivo do agente é maximizar as recompensas ao longo do tempo, aprendendo por meio de tentativa e erro qual ação tomar em cada situação.

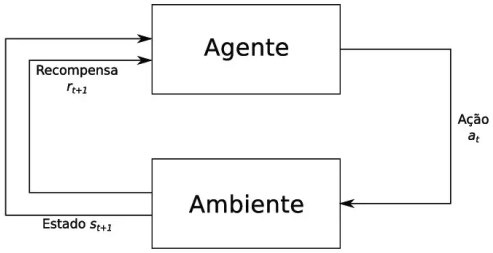


Figura 3. O Agente e seu ambiente.[Ayushi Singh et al. 2020]

2.4. NeuroEvolution of Augmenting Technologies

NEAT[Stanley 2012] é um algoritmo de aprendizado de máquina evolutivo baseado em algoritmos genéticos. Em vez de usar o tradicional backpropagation, o NEAT aplica a neuroevolução, permitindo a evolução de redes neurais por meio da mutação de nós, conexões e herança das redes mais aptas em novos indivíduos. O algoritmo também permite a existência de diferentes fenótipos, ou simplesmente ”espécies”de redes neurais, que competem entre si para produzir descendentes mais aptos. Para garantir a sobrevivência das espécies mais aptas, o NEAT adiciona um número de inovação a cada gene, garantindo que as redes já testadas não se repitam e permitindo a otimização das redes existentes.Aqui estão os principais conceitos do NEAT:

- **Codificação Genética:** A codificação genética é o processo de representar os cromossomos em um AG. O NEAT usa um esquema de codificação direta em que cada genoma consiste em uma lista de genes de nó e uma lista de genes de conexão. Os genes de nó indicam todos os nós de entrada, ocultos e de saída que podem ser conectados, enquanto os genes de conexão armazenam as informações de conexão e um número de inovação para marcações históricas. Ao fazer isso, o NEAT pode alinhar os genes correspondentes rapidamente durante o processo de cruzamento.

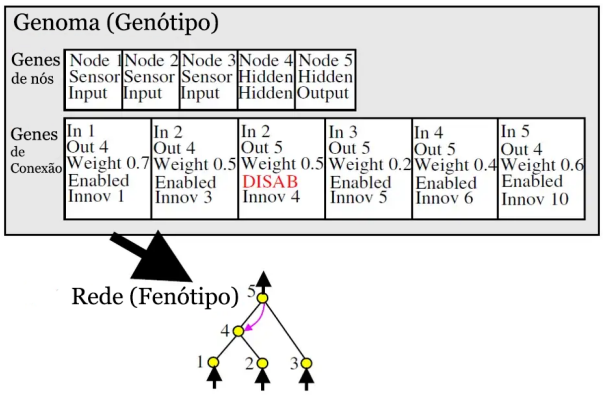


Figura 4. Exemplo de Codificação genética. [Cordeiro et al. 2019]

- **Mutação:** NEAT cresce e evolui a topologia por mutação estrutural. A mutação pode ser adicionar um novo nó em uma conexão antiga ou adicionar uma nova conexão entre dois nós desconectados. Portanto, o NEAT é capaz de melhorar a diversidade do genoma, explorar o espaço de solução e evitar mínimos locais. Além disso, a NEAT garante que diferentes tipos de “espécies” podem coexistir, até que possam competir entre si para produzir descendentes novos e mais aptos. Para garantir que as espécies mais aptas sobrevivam, as redes já testadas não se repetem no futuro e as redes existentes se otimizam, o NEAT adiciona um número de inovação a cada gene que funciona como um marcador histórico.

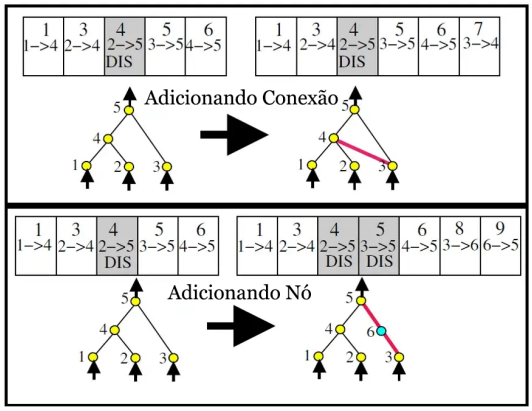


Figura 5. Exemplo de mutação. [Cordeiro et al. 2019]

- **Marcas Históricas:** Marcas históricas é o processo de rastreamento de genes. O NEAT usa um número de inovação global que representa uma cronologia de um gene no sistema para realizar marcações históricas. Um novo número de inovação será atribuído a esse gene sempre que um novo nó ou conexão for criado. Durante o cruzamento, a prole escolhe aleatoriamente genes com o mesmo número de inovação (genes correspondentes) de qualquer um dos pais e herda aqueles com número de inovação diferente (genes disjuntos ou em excesso) do pai mais apto. Consequentemente, o NEAT garante que o cruzamento seja significativo e resolva a convenção concorrente , uma situação em que pais com fenótipo semelhante produzem descendentes danificados.

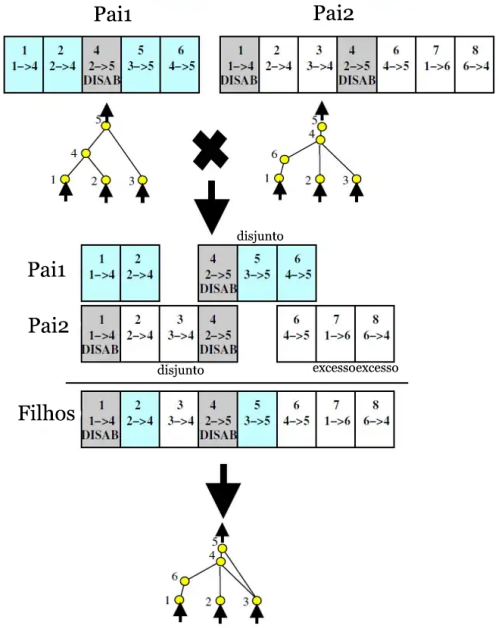


Figura 6. Exemplo de Marcações Históricas. [Cordeiro et al. 2019]

- **Especiação:** Uma inovação topológica geralmente vem com aptidão reduzida e requer tempo para otimizar seu desempenho. No entanto, se competir diretamente com a população geral, uma nova topologia provavelmente será eliminada antes

de atingir sua melhor aptidão. É por isso que a especiação desempenha um papel crítico no NEAT. O NEAT mede a distância de compatibilidade entre dois genomas. A compatibilidade é calculada por uma combinação linear das diferenças médias de peso de genes correspondentes e o número de genes em excesso e disjuntos . Os genomas são agrupados em diferentes espécies com base em um limite de compatibilidade. Ao fazer isso, cada genoma compete com aqueles dentro do mesmo nicho. Portanto, uma nova espécie será protegida.

$$\delta = \frac{c_1 E}{N} + \frac{C_2 D}{N} + c_3 \cdot \bar{W}$$

$\boldsymbol{\delta}$: distância de compatibilidade

$\boldsymbol{E}$: número de genes em excesso

$\boldsymbol{D}$: número de genes disjuntos

$\boldsymbol{W}$: diferenças de peso médio de genes correspondentes

$\boldsymbol{c_1, c_2, c_3}$: importância de cada fator

$\boldsymbol{N}$: número de genes no genoma maior

Figura 7. Equação para distância de compatibilidade. [Cordeiro et al. 2019]

- **Minimizando a dimensionalidade:** o NEAT sempre começa com uma população uniforme e sem nós ocultos. Uma nova estrutura é introduzida por mutação estrutural e protegida por especiação. Então a avaliação de adequação determina se a inovação é útil. Assim, o NEAT aumenta a complexidade apenas quando necessário e, assim, diminui o tempo de treinamento.

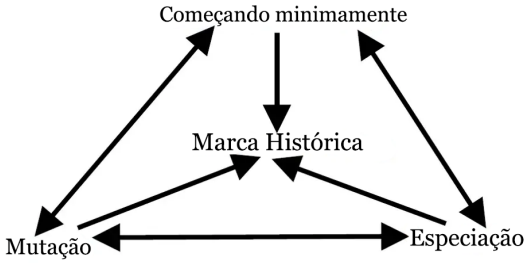


Figura 8. Dependências entre componentes. [Cordeiro et al. 2019]

Resumindo, o NEAT utiliza marcações históricas para identificar genes e evitar convenções concorrentes, de modo que um cruzamento significativo possa acontecer e menos análise topológica seja necessária. Separa a população em espécies com base na distância de compatibilidade, de modo que a competição ocorra principalmente dentro do mesmo nicho e a inovação seja protegida. Parte da estrutura mais simples e cresce apenas conforme necessário por meio de mutação, para que seja mais rápido encontrar uma solução.

3. Metodologia

Três componentes são essenciais para este trabalho: cálculo da função de aptidão, apresentado na seção 3.1; como expor o cenário ao agente, apresentado na seção 3.2; e configurações de fenótipo, apresentadas na seção 3.3.

3.1. Função de Aptidão

Para calcular a aptidão do agente são utilizados três componentes, denominados Componentes de Aptidão (CA):

- Altura do Pássaro (AP): coordenada y do agente em relação ao chão.
- Pontuação: calculado cada vez que o agente passa pelo cano.
- Fator Y(ΔY): É calculado pela diferença entre a coordenada y do agente e o ponto médio do cano superior e inferior.

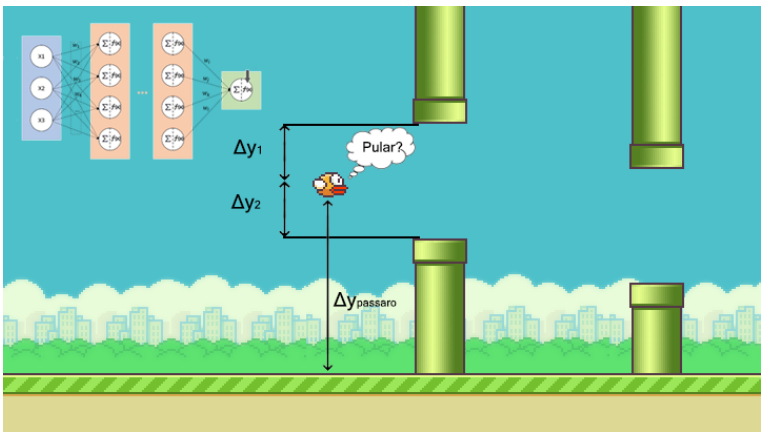


Figura 9. Demostração da entrada e saída. [Danny Zhu 2020]

Para uma dada posição do pássaro, digamos (x, y), existem duas ações que o agente pode fazer. Ou o pássaro bate as asas ou não bate as asas [Selvan and Game 2022]. A altura do pássaro desempenha um papel fundamental, pois permite penalizar o desempenho do agente na função de aptidão com um valor que expressa a distância que o agente estava do objetivo, a passagem entre canos, antes de falhar. Os valores só serão considerados quando o agente falhar, definido pela colisão com qualquer parte do cenário. Quando isso ocorre, a interação com o ambiente cessa e o desempenho do agente é medido. O Fator Y é utilizado para transmitir um valor de qualidade para o cálculo da função de aptidão, pois diferencia os agentes que falharam no mesmo par de canos, mas em alturas diferentes. O objetivo é garantir que os agentes mais próximos da passagem sejam considerados melhores do que outros mais distantes, algo que não seria possível se a atuação do agente fosse baseada apenas na AP e Pontuação.

Esses três CAs foram combinados em uma única expressão, chamada de função de aptidão do cenário (FAC):

$$FAC = \alpha \times AP + \beta \times Pontuacao + \lambda \times \Delta Y \tag{1}$$

Essa expressão mostra que a aptidão do cenário de um agente é uma combinação linear de CAs e três pesos constantes, α, β e λ, que regulam a importância de cada componente na aptidão. O objetivo do agente é então percorrer a maior distância possível, obter

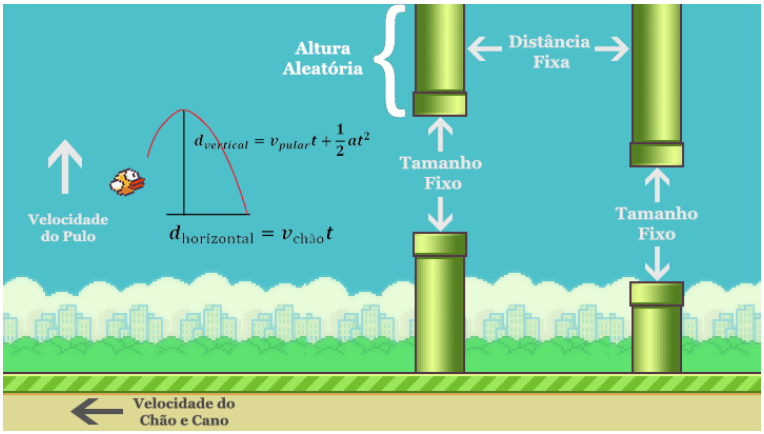


Figura 10. Design do jogo Flappy Bird. [Danny Zhu 2020]

a maior pontuação e ficar mais tempo vivo. Com a Função de adaptação devidamente calculada, o NEAT soma o valor com o Bias e aplica a função de ativação, assim obtendo o output. Falaremos com mais detalhes sobre o output na seção 3.3.

3.2. Cenários

O principal objetivo em relação à forma como o cenário é exposto ao agente durante o treinamento é apresentar novos desafios e ainda preservar a capacidade do algoritmo de convergir em um curto espaço de tempo. Três cenários são representados:

I Cenário e objetos estáticos com velocidade constante.
II Cenário estático com objetos movendo e velocidade constante.
III Cenário e objetos móveis com velocidade incremental.

3.3. Fenótipo de Rede

O fenótipo do agente é representado por uma RNA que possui três neurônios de entrada, um neurônio de saída e com uma estrutura interna que só será definida após o treinamento pelo algoritmo. As entradas são os CAs conforme mostrado na seção 3.1 e a saída da rede é a probabilidade de realizar um salto.

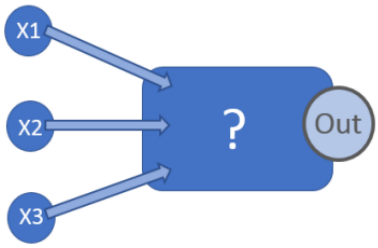


Figura 11. Rede Neural com uma estrutura interna inicialmente desconhecida.
fonte:[Cordeiro et al. 2019]

Quando o agente recebe a informação dada pelo ambiente (Altura do Pássaro, Pontuação e Fator Y), ela é processada pela rede, gerando uma probabilidade de execução de um salto. Essa probabilidade determinará a ação a ser executada de acordo com a regra:

Ação = Pular **if**(out ≥ 0.7) **else** Nenhum.

Em rede com classificação binária, usualmente o valor escolhido é de 0.5, porém visando deixar o treinamento um pouco mais complexo, usaremos o valor 0.7.

3.4. Parâmetros e Ferramentas

A implementação do NEAT em python é bastante simples, pois existe um módulo bem desenvolvido[Yash Shah 2022]. Na configuração dos parâmetros do NEAT, foram estabelecidos os seguintes valores, que foram utilizados para reproduzir este trabalho:

- **População:** foi utilizada uma população inicial com 10 indivíduos. Devido o cenário (I) ser relativamente simples, se o numero for muito grande, tende a convergir muito rápido.
- **Elitismo:** foi escolhido um elitismo de 20%.
- **Taxa de mutação:** foi utilizado o valor de 5%, o que fazia com que as conexões não fossem ativadas imediatamente à medida que a topologia aumentava, eram ativadas gradativamente.
- **Peso e Bias:** a média inicial de geração de pesos e bias foi de 0% e 1%, respectivamente, com desvio padrão de 100% em ambos. Esses valores permitem uma distribuição um pouco mais variada quando os pesos são gerados, aproximando-se assim das topologias de melhor desempenho.
- **Probabilidades de adicionar ou remover conexões:** a probabilidade de adicionar conexões e a probabilidade de remover conexões foram definidas para 50% para ambas.
- **Probabilidades para adicionar ou remover nós:** a probabilidade de adicionar nós foi definida em 20% e a probabilidade de remover nós foi definida em 30%, como nos parâmetros acima, e eles têm uma explicação semelhante.
- **fitness threshold:** foi utilizado um limiar de 400 para dizer se um agente já aprendeu como passar pelos obstáculos do cenário.
- **Função de ativação:** foi utilizado a função Tangente Hiperbólica e com uma taxa de 10% de mutação para Sigmoide.

4. Resultados

A implementação do algoritmo não requer dados históricos ou qualquer conjunto de dados. O algoritmo faz uso dos dados sensoriais percebidos do ambiente pelo agente à medida que o programa é executado. As entradas para o algoritmo são a distância vertical do agente até o chão, a distância vertical do agente do cano superior e a distância vertical do agente do cano inferior. A saída do algoritmo é a ação que o agente deve realizar, ou seja, pular ou nenhuma (cair devido à gravidade).

4.1. Fitness

Devido a baixa complexidade do cenário (I), as arquiteturas lineares eram suficientes para captar as dinâmicas da simulação e entre pesos aleatórios iniciados, alguma acabava por ser a solução do ótimo ou bem próximo, bastando apenas pequenos ajustes. Em casos que a rede não iniciava com a solução ótima, levava em média 3 gerações para encontrar a solução do jogo com apenas uma espécie, pois o tempo não era suficiente para que ocorressem mutações que formassem outras espécies. Diante disso, os dados eram desinteressantes e não mostraremos os gráficos do resultado para esse cenário.

No caso dos cenários (II) e (III) que possuíam um grau de complexidade maior, as simulações tiveram médias de convergência e variância parecidas conforme mostrado na figura 12, intuindo que provavelmente algumas simulações, se o threshold do fitness fosse sensivelmente aumentado, não convergiriam na mesma geração. Isso mostra que além de aprender regras básicas do jogo também existe uma curva de qualidade a sobre a jogatina, a qual diferentes agentes artificiais atingem diferentes níveis. Em média o cenário (II) levou de 8 a 12 gerações para encontrar solução, enquanto o cenário (III) levou de 14 a 17 gerações.

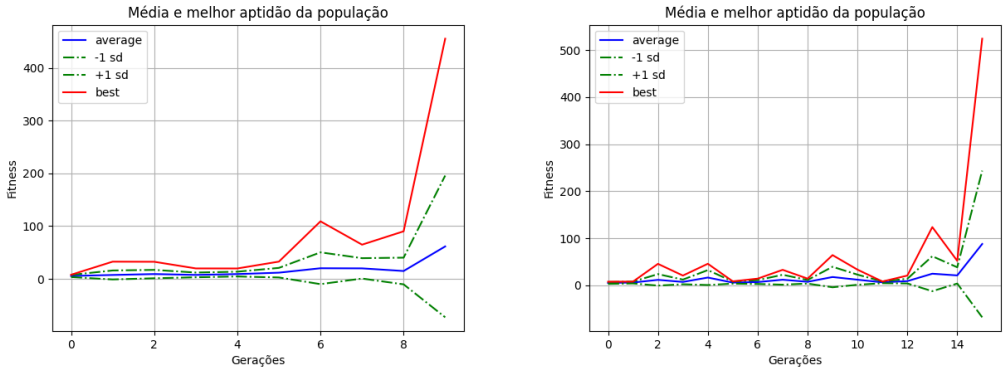


Figura 12. Resultado de uma simulação para o cenário II e III.
fonte:[Próprio autor]

O fitness é calculado a partir das recompensas ou penalidades que o aprendizado por reforço aplica sobre as ações do agente. O agente recebera penalidade de -1 ponto para colisões com os objetos do ambiente; Uma recompensa de 0.1 ponto para cada frame que permanecer vivo; Uma recompensa de 5 pontos por passar entre o canos.

4.2. Especiação

A figura 13 mostra o gráfico de especiação, no qual o eixo x informa as gerações e o eixo y o tamanho da espécie que é no máximo 10. No tangente ao número de espécies concluímos que quanto mais gerações ocorressem, maior seria o número de novas espécies devido aos operadores genéticos (seleção, cruzamento, mutação, etc.).

O cenário (I) com apenas uma espécie foi suficiente para convergir e resolver o jogo, apenas uma única a cor seria mostrada. Por outro lado, os cenários (II) e (III), desenvolveram mais espécies para lidar com diferentes obstáculos apresentados. Todavia, nem sempre é necessário desenvolver espécies para encontrar a solução, em alguns testes o NEAT conseguiu convergir para solução de ambos os cenários com apenas uma espécie.

4.3. Topologia

Também pudemos observar a variabilidade nas estratégias aprendidas pelo NEAT em cada simulação e cenário. Para cenários de maior complexidade as topologias começam a variar e apresentar diferentes estratégias. A arquitetura da rede do melhor agente também começa a apresentar um número maior de camadas afim de capturar relações não lineares presentes. O fenótipo do melhor agente da última geração dos cenários (II) e (III) respectivamente é mostrado na figura 14.

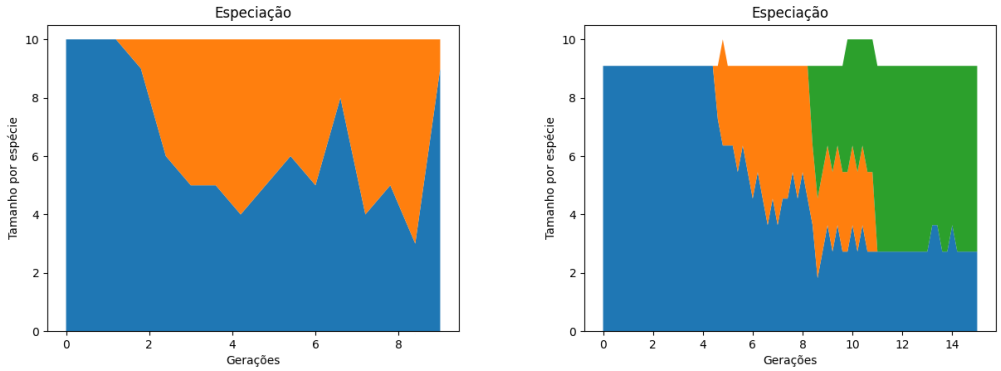


Figura 13. Gráfico de especiação de uma simulação para os cenários II e III.
fonte:[Próprio autor]

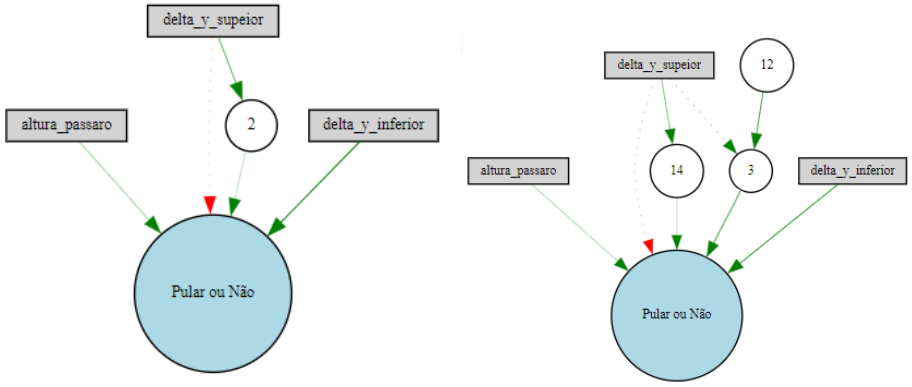


Figura 14. Topologias da rede do cenário I e II.
fonte:[Próprio autor]

O fenótipo é um perceptron, e um modelo muito simples de uma rede neural seria a do cenário (I), com somente três entrada conectadas a única saída. Já para o cenário (II), o NEAT adicionou um novo nó, provavelmente para lidar com o obstaculo que antes não havia, que são os canos se movendo. Já para o cenário (III) temos uma rede um pouco mais elaborado, foi adicionado um novo nó de entrada, possivelmente para representar o novo obstaculo que é a velocidade incremental.

5. Conclusão

Neste artigo, foi apresentado uma estratégia de treinamento eficiente para gerar agentes capazes de alcançar excelentes pontuações no jogo Flappy Bird usando três tipos diferentes de cenários com diferentes tipos de obstáculos. Com o objetivo de manter o agente vivo em um ambiente com pares ilimitados de canos, foi utilizado rede neural, algoritmo genético e aprendizado por reforço durante os treinos. A estratégia do fitness utilizada garantiu que o objetivo do agente em diferentes cenários fosse alcançada.

Usando uma população de apenas 10 indivíduos, o algoritmo foi capaz de convergir para um comportamento ótimo aproximadamente quatorze gerações. Nesse ponto, um agente é capaz de jogar o jogo indefinidamente. Como trabalhos futuros, essa estratégia de treinamento pode ser testada em jogos de plataforma simples que apresentam algum tipo

de repetição de ação ao longo de suas etapas, como por exemplo, o jogo Dino do Google. Em caso de jogos mais complexos como Sonic e Mário, precisaria reajustar os parâmetros e através buscas realizadas na internet mostraram que o NEAT se saiu muito bem para esses casos, em especial aos canais [Hashtag Programação 2021] e [Tech With Tim 2019], que abordam muito bem sobre o assunto.

Referências

Ayushi Singh, N. T., Sahil, K., and Chaudhary, V. (2020). Flappy bird implementation using ai. In *FLAPPY BIRD IMPLEMENTATION USING AI*, pages 1–6. International Journal For Technological Research In Engineering.

Cordeiro, M. G., Serafim, P. B. S., Nogueira, Y. L. B., Vidal, C. A., and Neto, J. B. C. (2019). A minimal training strategy to play flappy bird indefinitely with neat. In *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, pages 21–28. IEEE.

Danny Zhu (2020). How i built an intelligent agent to play flappy bird. Disponível em: `https://medium.com/analytics-vidhya/how-i-built-an-ai-to-play-flappy-bird-81b672b66521`. Acesso em: 20 de novembro 2022.

Hashtag Programação (2021). Como criar uma inteligência artificial. Disponível em: `https://www.youtube.com/watch?v=fR7835SFx-o&ab_channel=HashtagPrograma`. Acesso em: 19 de novembro 2022.

Selvan, J. P. and Game, P. S. (2022). Playing a 2d game indefinitely using neat and reinforcement learning. *arXiv preprint arXiv:2207.14140*.

Stanley, K. O. (2012). Evolving neural networks. In *Proceedings of the 14th annual conference companion on Genetic and evolutionary computation*, pages 805–826.

Tech With Tim (2019). Python flappy bird ai. Disponível em: `https://www.youtube.com/watch?v=MMxFDaIOHsE&ab_channel=TechWithTim`. Acesso em: 19 de novembro 2022.

Victor de Oliveira (2018). Neuroevolução: uma aplicação em aprendizado por reforço. Disponível em: `https://vtoliveira.github.io/projetos/neuroevolucao`. Acesso em: 20 de novembro 2022.

Yash Shah (2022). Flappy bird ai project in python. Disponível em: `https://bytesofintelligence.co.uk/flappy-bird-ai-project-in-python/`. Acesso em: 27 de novembro 2022.